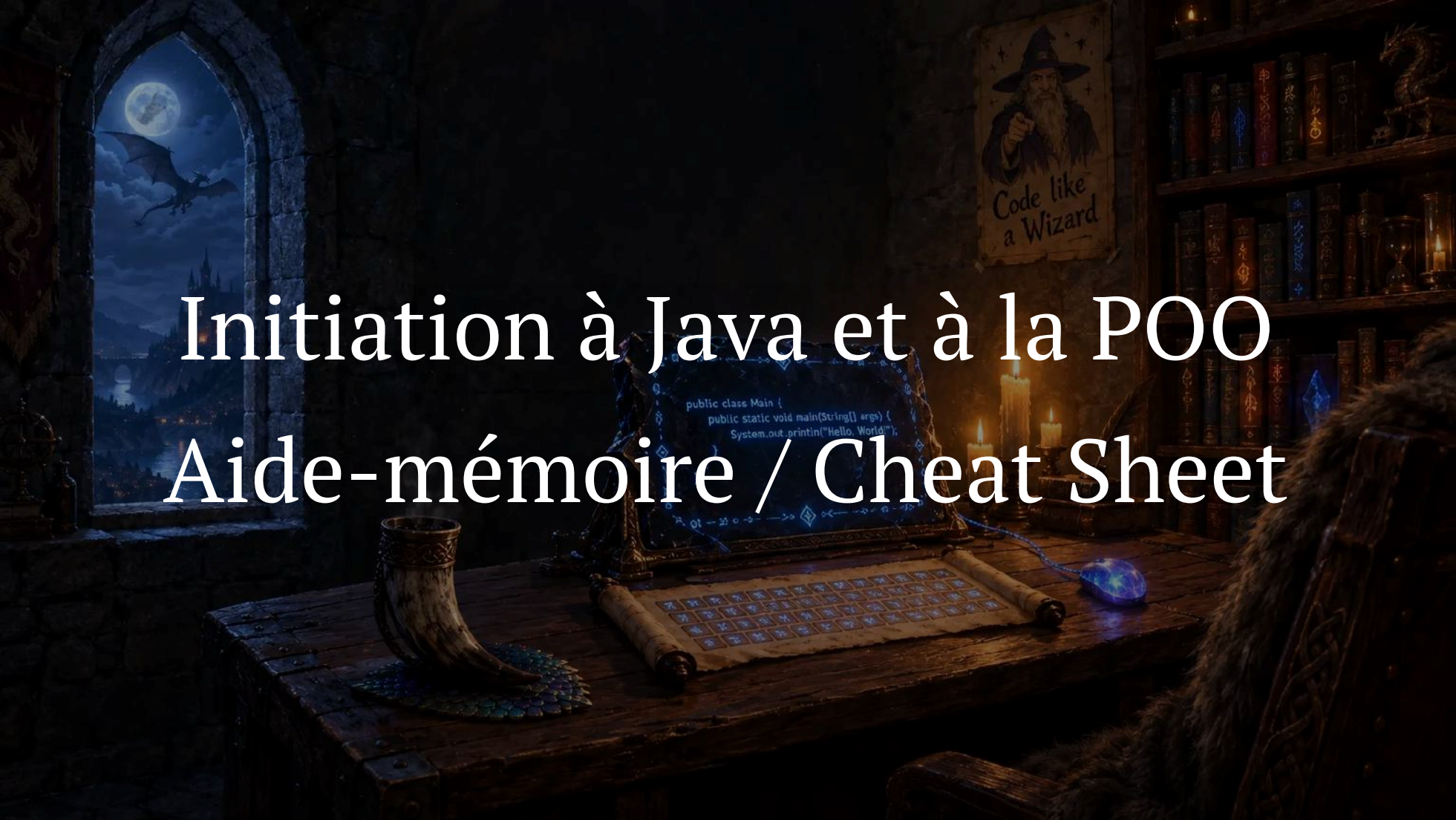


Initiation à Java et à la POO
Aide-mémoire / Cheat Sheet



Nathaniel Vaur Henel

L'auteur

Auteur

👨 Nathaniel Vaur Henel 🤘

- 👨‍🏫 Formateur
- 👨‍💻 Tech leader
- 🧉 Expert Java
- 💻 20 ans d'expérience en développement, principalement en Java
- 📜 Créateur de contenus pour dévs : dev-sensei.digicrafters.fr
- Discord : `@nathanielvaurhenel`



framagit.org/nathvh/me



dev-sensei.digicrafters.fr



linkedin.com/in/nathaniel-vaur-henel

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

1. 📁 Structure d'un fichier Java

```java
// 1. Déclaration du package (= dossier logique)
package com.monprojet.utils;
// 2. Les imports (bibliothèques externes)
import java.util.ArrayList;
import java.util.List;
// 3. La classe principale (un seul fichier = une seule classe publique)
public class MaClasse {
    // Attributs (variables de la classe)
    private String nom;
    private int age;
    // Constructeur(s)
    public MaClasse(String nom, int age) {
        this.nom = nom;
        this.age = age;
    }
    // Méthodes
    public String getNom() {
        return nom;
    }
}
```

⚠️ **Règle d'or** : Le nom du fichier `.java` doit être **identique** au nom de la classe publique. Exemple : `MaClasse.java` contient `public class MaClasse`.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

2. 🏷️ Conventions de Nommage

Élément	Convention	Exemple
Classe	`PascalCase` (majuscule à chaque mot)	`CompteBancaire`, `VoitureSport`
Interface	`PascalCase` (souvent un adjectif)	`Serializable`, `Comparable`
Variable	`camelCase` (minuscule, puis majuscule)	`nomUtilisateur`, `prixTotal`
Méthode	`camelCase` (commence par un verbe)	`calculerTotal()`, `getNom()`
Constante	`SNAKE_CASE_MAJUSCULE`	`MAX_TENTATIVES`, `PI`
Package	tout en minuscules, sans tirets	`com.monprojet.utils`

```
public class CompteBancaire { // ✅ Correct
    private static final double TAUX_INTERET = 0.05;
    private double soldeActuel;
    public double calculerInterets() {
        return soldeActuel * TAUX_INTERET;
    }
}
public class compte_bancaire { // ❌ Incorrect
    private static final double tauxInteret = 0.05;
    private double SoldeActuel;
    public double Calcul_interets() { ... }
}
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

3. 🗂️ Déclaration des Variables

Types primitifs courants

```
int age = 25;             // Entier (-2 milliards à +2 milliards)
long population = 8000000000L; // Grand entier (ne pas oublier le L)
double prix = 19.99;      // Décimal (précision élevée)
float taux = 3.14f;       // Décimal (moins précis, ne pas oublier le f)
boolean estActif = true;  // true ou false uniquement
char initiale = 'A';      // Un seul caractère (guillemets simples)
```

Types objets courants

```
String prenom = "Alice";          // Chaîne de caractères (guillemets doubles)
String vide = null;               // null = absence de valeur

int[] notes = {12, 15, 18};      // Tableau de tailles fixe
List<String> prenoms = new ArrayList<>(); // Liste dynamique
```

> 💡 **Bonne pratique** : Déclarez vos variables aussi près que possible de leur utilisation, pas toutes en haut de la méthode.

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

4. 🔒 Modificateurs d'Accès

Modificateur	Visible depuis
`public`	Partout
`protected`	Même package + sous-classes
(rien)	Même package uniquement
`private`	Classe uniquement

```
public class Personne {
    private String nom;       // Accessible uniquement dans Personne
    protected int age;        // Accessible dans Personne et ses sous-classes
    public String pays;       // Accessible partout (déconseillé pour les attributs)
}
```

💡 **Règle de base** : Les **attributs** sont toujours `private`. On y accède via des **getters/setters**.

5. 🔁 Getters & Setters

```java
public class Produit {
    private String nom;
    private double prix;

    // Getter : retourne la valeur
    public String getNom() {
        return nom;
    }

    // Setter : modifie la valeur (avec validation possible)
    public void setPrix(double prix) {
        if (prix >= 0) {
            this.prix = prix;
        }
    }

    // Cas particulier : getter pour un boolean
    public boolean estEnStock() {
        return enStock;
    }
}
```

6. 📐 Indentation & Mise en forme

```
// ✅ Accolade ouvrante SUR la même ligne (style Java standard)
public void maMethode() {
    if (condition) {
        // Indentation de 4 espaces (ou 1 tabulation)
        faireQuelqueChose();
    } else {
        faireAutreChose();
    }
}
// ❌ À éviter
public void maMethode()
{    // Accolade sur la ligne suivante (style C#, pas Java)
if(condition){faireQuelqueChose();} // Tout sur une ligne
}
```

Règles de base :

- Indentation : **4 espaces** (configurez votre IDE)
- Une seule instruction par ligne
- Ligne vide entre les méthodes
- Longueur de ligne recommandée : **max 120 caractères**

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

7. Commentaires

```
// Commentaire sur une ligne : pour expliquer une ligne complexe

/*
 * Commentaire sur plusieurs lignes :
 * pour expliquer un bloc de code.
 */

/**
 * Commentaire Javadoc : pour documenter une classe ou méthode.
 * Apparaît dans l'IDE au survol.
 *
 * @param montant  Le montant à déposer (doit être positif)
 * @return         Le nouveau solde du compte
 */
public double deposer(double montant) {
    solde += montant;
    return solde;
}
```

> 💡 **Conseil** : Commentez le **pourquoi**, pas le **quoi**. Un code bien nommé se lit tout seul.

```
// ❌ Inutile : "on ajoute 1 à i"
i = i + 1;

// ✅ Utile : pourquoi on skip le premier élément
i = i + 1; // On ignore l'en-tête du tableau
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

8. 🏗️ Structure d'une Classe Complète (Ordre recommandé)

```java
public class Etudiant {
  // 1️⃣ Constantes (static final)
  public static final int AGE_MINIMUM = 16;
  // 2️⃣ Attributs statiques
  private static int nombreEtudiants = 0;
  // 3️⃣ Attributs d'instance
  private String nom;
  private String prenom;
  private double moyenne;
  // 4️⃣ Constructeur(s)
  public Etudiant(String nom, String prenom) {
    this.nom = nom;
    this.prenom = prenom;
    this.moyenne = 0.0;
    nombreEtudiants++;
  }
  // 5️⃣ Méthodes publiques
  public void ajouterNote(double note) {
    // ...
  }
  // 6️⃣ Getters et Setters
  public String getNom() {
    return nom;
  }

  public void setNom(String nom) {
    this.nom = nom;
  }

  // 7️⃣ Méthodes héritées (toString, equals...)
  @Override
  public String toString() {
    return prenom + " " + nom + " (moyenne: " + moyenne + ")";
  }

  // 8️⃣ Méthodes privées (utilitaires internes)
  private double calculerMoyenne() {
    // ...
  }
}
```

9. ⚡ Bonnes Pratiques Essentielles

✅ À faire

```
// Utiliser des noms parlants
double prixAvecTVA = prix * 1.20; // ✅
double x = p * 1.20;              // ❌

// Éviter les "magic numbers" : utiliser des constantes
static final int JOURS_PAR_SEMAINE = 7;
int totalJours = semaines * JOURS_PAR_SEMAINE; // ✅
int totalJours = semaines * 7;                 // ❌

// Vérifier les cas limites
public double diviser(double a, double b) {
    if (b == 0) {
        throw new IllegalArgumentException
            ("Division par zéro impossible");
    }
    return a / b;
}

// Préférer List à tableau quand la taille peut varier
List<String> noms = new ArrayList<>(); // ✅
String[] noms = new String[100]; // ❌ (taille au hasard)
```

❌ À éviter

```
// Ne jamais comparer des Object/String avec ==
if (prenom == "Nath") { ... } // ❌ Compare les références
                              // , pas le contenu
if (prenom.equals("Nath²")) { ... }    // ✅

// Ne pas laisser des catch vides
try {
    // ...
} catch (Exception e) {
    // ❌ Silencieux = bug invisible
}

try {
    // ...
} catch (Exception e) {
    System.err.println("Erreur : " + e.getMessage()); // ✅
}

// Ne pas retourner null sans raison,
// préférer une exception ou une valeur par défaut
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

10. 📦 Organisation des Packages

```
src/
└── com/
    └── monprojet/
        ├── model/        → Classes représentant les données (Etudiant, Produit...)
        ├── service/      → Logique métier (EtudiantService...)
        ├── utils/        → Outils réutilisables (DateHelper, StringUtils...)
        └── Main.java     → Point d'entrée du programme
```

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

🎯 Résumé Rapide

Règle	À retenir
Nommage	`PascalCase` classes · `camelCase` méthodes/variables · `MAJUSCULES` constantes
Accès	Attributs `private` + getters/setters `public`
Fichier	1 fichier = 1 classe publique, nom identique
Accolades	Ouvrante sur la même ligne
Indentation	4 espaces
Commentaires	Javadoc pour les méthodes publiques
String/Object	Toujours `.equals()`, jamais `==`
Magic numbers	Remplacer par des constantes nommées

© **Nathaniel Vaur Henel** pour DigiCrafters/Dev Sensei. https://dev-sensei.digicrafters.fr - Partageable et publiable, non réutilisable, avec mention de l'auteur. **CC BY-ND 4.0**

🤝 Merci de ton attention ! 🤝

Reste curieux 🧐 et amuse-toi ! 🎮 🎉

Cet aide-mémoire t'est offert par

👨 Moi-même, Nathaniel Vaur Henel 🤘

mon site de partage autour du dév : dev-sensei.digicrafters.fr

mon Framagit perso, GitLab libre & français : framagit.org/nathvh/me

mon GitHub perso, mais inactif : github.com/Nathaniel-Vaur-Henel

mon LinkedIn, pour suivre mon parcours pro : linkedin.com/in/nathaniel-vaur-henel

mon Discord pro, pour échanger ou me poser des questions : `@nathanielvaurhenel`